TFM / BIGSCHOOL

Pasarela IoT TCP escalable

Manual completo del proyecto

Funcionamiento, arquitectura y protocolo

Instalación, operación y pruebas verificadas

Seguridad, decisiones y preparación de la entrega

Máster en Desarrollo con IA E.1

Versión de referencia v1.0.0 - 15 de octubre de 2026

Proyecto individual con dispositivos y operaciones ficticios.

Documento consolidado a partir del código y la evidencia del repositorio.

Índice

Guía técnica y de uso de la versión publicada. Las entradas del índice y los enlaces del documento son navegables. El manual distingue requisitos, comportamiento implementado y resultados históricos.

1. Visión general del proyecto 3

2. Requisitos y alcance implementado 4

3. Arquitectura y componentes 5

4. Recorrido completo de una petición 6

5. Contrato del protocolo IoT v1 7

6. Acciones y ejemplos funcionales 8

7. Códigos de resultado y tratamiento de errores 9

8. Consistencia, concurrencia e idempotencia 10

9. Base de datos y auditoría 11

10. Seguridad y gestión de secretos 12

11. Instalación y primera ejecución 13

12. Uso del simulador 15

13. Operación, escalado y salud 16

14. Validación funcional y resultados de carga 18

15. Fallos, recuperación y copia de seguridad 19

16. Reproducción de pruebas y diagnóstico 20

17. Publicación, CI y versión de entrega 21

18. Decisiones técnicas y uso de IA 22

19. Límites, evolución y preparación de la entrega 23

20. Referencia técnica y fuentes 24

No se incluyen claves operativas, contraseñas ni datos financieros reales. Las cifras de las pruebas están vinculadas a sus condiciones y no constituyen un compromiso de servicio.

1. Visión general del proyecto

La Pasarela IoT TCP escalable es el Trabajo Fin de Máster del Máster en Desarrollo con IA E.1 de BigSchool. Resuelve un escenario de comunicación entre dispositivos y un servidor: mantiene conexiones TCP persistentes, autentica cada dispositivo, valida las peticiones y registra sus operaciones en PostgreSQL. Incluye un simulador capaz de representar cientos o miles de dispositivos configurables.

El proyecto nace de la experiencia del autor desarrollando una pasarela de pagos. Esta implementación es independiente y utiliza un protocolo propio, datos ficticios y sumas matemáticas sencillas. No procesa pagos, no maneja tarjetas y no integra bancos. La escala 100 de los números es una convención didáctica; no representa una moneda.

Objetivo y restricciones

La prioridad es una primera versión completa, explicable y entregable con rapidez. El autor trabaja con C++, C#, .NET y Git, dispone de una dedicación semanal variable y fijó un presupuesto máximo total de 100 EUR. No se han contratado servicios para el proyecto ni se requiere una API de IA de pago para ejecutarlo. Las suscripciones personales a ChatGPT y Gemini no forman parte de un coste operativo de la aplicación.

El equipo disponible incluye un Synology DS920+ y un CHUWI LarkBox X con Intel N150, 16 GB de RAM, SSD de 512 GB y Proxmox. Las pruebas publicadas se realizaron en el PC con Docker Desktop, no en esos equipos. No se ha validado un despliegue en NAS, Proxmox ni cloud.

Estado de referencia de este manual

La documentación describe la versión publicada v1.0.0, cuyo commit de cierre es 245b834. El código y la configuración de CI se comprobaron en el commit f6ef8b9; después se actualizaron enlaces y evidencias documentales. La fecha de corte es el 15 de octubre de 2026, con referencia de calendario Europe/Madrid. Los sellos de tiempo del protocolo y de las evidencias utilizan UTC cuando así lo indican.

La versión incluye dos servicios replicables de aplicación, una base independiente, balanceador, aprovisionamiento, simulador, pruebas y presentación. Los dos trabajos de GitHub Actions finalizaron correctamente. El autor preparará personalmente las diapositivas definitivas a partir de la guía PDF. No tiene previsto grabar vídeo. Quedan la revisión final, el acceso del evaluador y el envío al campus; debe aclararse que el formulario actual exige URL de vídeo aunque las explicaciones lo describían como opcional.

Este manual consolida requisitos, implementación y resultados. Cuando la especificación inicial expresa una intención que la versión actual no cumple, prevalecen el código y las precisiones de este documento. No se incluyen claves, contraseñas ni información privada de un despliegue.

2. Requisitos y alcance implementado

Necesidad del proyecto	Solución de la versión v1.0.0
TCP con conexión persistente	Sockets asíncronos, SslStream y una sesión por conexión.
Dispositivos identificables	Fabricante, modelo y serie conservados como cadenas con ceros iniciales.
Validación de estructura	Protocolo ASCII versionado, campos delimitados y tamaño acotado.
Comprobar el registro del dispositivo	Servicio registry consulta PostgreSQL y verifica su estado y HMAC.
Identificar el comercio	Respuesta autenticada con identificador de nueve dígitos, según resultado.
Registrar actividad y mensajes	Tablas de conexiones, intentos, peticiones, operaciones y totales.
Ejecutar acciones sencillas	Inicialización, suma exacta y consulta de totales acumulados.
Proteger las comunicaciones	TLS externo y HMAC-SHA-256 por dispositivo.
Replicar dos componentes	gateway y registry con Docker Compose y HAProxy.
Simular carga concurrente	Cliente configurable con rampa, pausas, sincronización y métricas.
Recuperar operaciones tras un fallo	Reconexión, reintentos acotados y deduplicación en PostgreSQL.
Facilitar la evaluación	Repositorio Git, instrucciones, evidencias, manual y guía de diapositivas; acceso del evaluador pendiente.

Qué significa «alta capacidad» en este proyecto

El requisito original se refiere a cientos de conexiones y operaciones concurrentes. Se han ensayado perfiles de 100, 500 y 1000 dispositivos preparados, además de una carga sostenida de 500. Un límite configurado no constituye una capacidad medida: por ejemplo, permitir 2000 conexiones por gateway no demuestra que 2000 clientes cumplan una latencia objetivo.

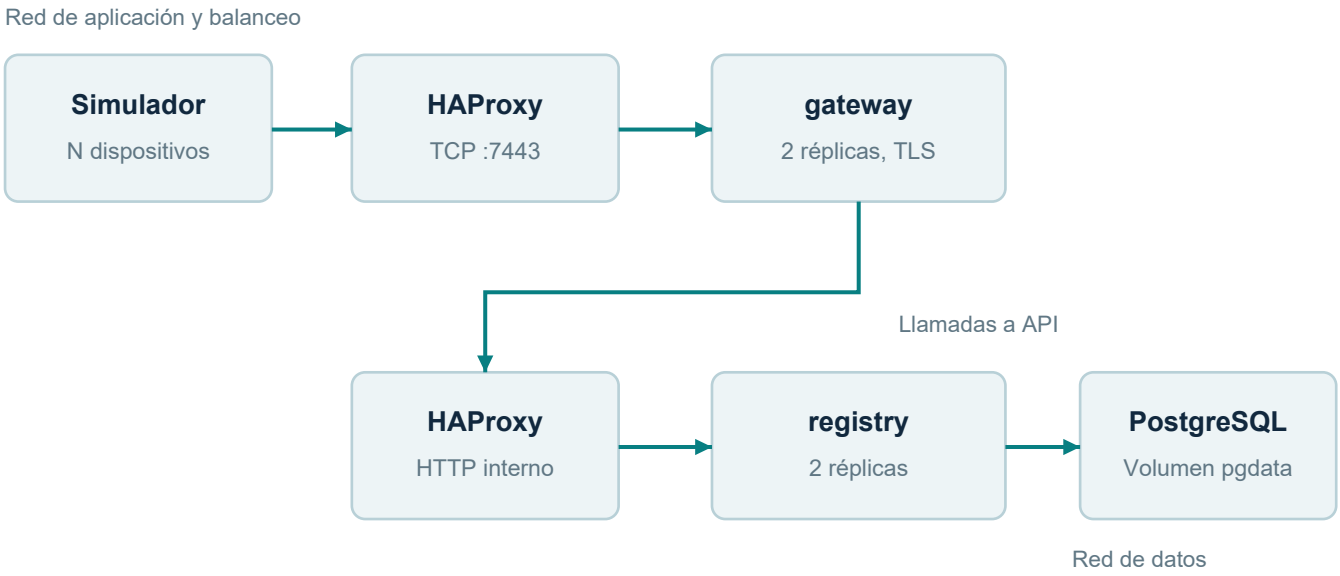
Replicar servicios permite repartir conexiones nuevas y tolerar determinadas caídas de aplicación. No se ha demostrado una mejora de velocidad al duplicar réplicas en la comparación exploratoria realizada. Se explican por separado capacidad, latencia y recuperación.

Fuera del MVP

No se incluyen interfaz gráfica de administración, pagos reales, Kubernetes, broker, caché distribuida, modelos de aprendizaje automático ni migración transparente de sesiones TCP. Tampoco hay autoescalado, calendario automático de réplicas, clúster PostgreSQL, redundancia del balanceador o del host. Estos elementos son ampliaciones posibles y no requisitos pendientes de una función central a medias.

3. Arquitectura y componentes

La solución conserva tres componentes funcionales: entrada TCP, registro/operaciones y base de datos. HAProxy es infraestructura adicional para distribuir tráfico. El simulador, el aprovisionador y las pruebas son herramientas auxiliares.



Solo la entrada TCP se publica en localhost. Las dos funciones de HAProxy comparten instancia.

La figura representa el perfil probado de dos gateways y dos registros. El mismo HAProxy dispone de una entrada TCP para dispositivos y otra HTTP interna para las peticiones del gateway. TLS termina en el gateway; el balanceador TCP transporta sus bytes. Solo se publica en el host la entrada 127.0.0.1:7443.

Responsabilidades y aislamiento

Componente	Responsabilidad y estado
gateway	Acepta TCP/TLS, delimita tramas, limita conexiones y trabajo activo, conserva el estado de las conexiones en memoria y devuelve respuestas. No accede a PostgreSQL ni recibe claves HMAC de dispositivos.
registry	Revalida la trama, consulta identidad y estado, verifica HMAC, ejecuta acciones y transacciones, firma respuestas y mantiene auditoría. El estado funcional reside en PostgreSQL.
db	Conserva comercios, dispositivos, conexiones, intentos, respuestas idempotentes, operaciones y totales en un volumen persistente.
proxy	Distribuye conexiones TCP con leastconn y peticiones HTTP con roundrobin entre destinos preparados. Resuelve réplicas mediante el DNS de Docker.
simulator	Abre conexiones, autentica mensajes, verifica resultados y totales, reconecta y genera informes JSON/CSV.
provision	Genera secretos y certificado de laboratorio, aplica el esquema inicial y aprovisiona datos ficticios.

Tecnologías y organización del código

La implementación usa C# con .NET 10, ASP.NET Core, Npgsql 10.0.0, PostgreSQL 18, Docker Compose, HAProxy 3.0 y xUnit. global.json selecciona el SDK 10.0.402 con rollForward latestFeature. Las versiones e imágenes concretas de las pruebas se registran en los metadatos; las etiquetas de imagen no están fijadas por digest en toda la configuración.

Iot.Protocol contiene formato, firma, FrameReader y contratos. Iot.Persistence contiene configuración, cifrado y esquema. Iot.Gateway e Iot.Registry implementan los dos servicios. Iot.Provisioning e Iot.Simulator son ejecutables auxiliares. Iot.Tests agrupa las pruebas de protocolo, PostgreSQL y red real. La solución es IotGateway.slnx.

4. Recorrido completo de una petición

El ejemplo siguiente explica una suma aceptada. El dispositivo ya dispone de una identidad, una clave HMAC y un certificado público de confianza generados para la demo.

1. El simulador abre TCP hacia proxy:7443 dentro de Docker. Desde el host, la entrada equivalente es localhost:7443. HAProxy asigna la nueva conexión a un gateway preparado.
2. El cliente y el gateway negocian TLS. El simulador comprueba el nombre y la huella SHA-256 del certificado esperado. El gateway registra la apertura por medio de la API interna.
3. El cliente envía una petición 001 firmada. registry verifica el registro, el secreto, el estado activo y el resto de reglas. Vincula el identificador de conexión a la identidad autenticada.
4. El servidor devuelve 000, el comercio asociado y READY, con HMAC. El cliente comprueba correlación y autenticidad.
5. El cliente crea una petición 002 con un identificador aleatorio nuevo, fecha UTC y los operandos. Calcula HMAC y escribe la línea completa con LF.
6. FrameReader reconstruye la línea aunque llegue en varios fragmentos o junto a otras líneas. El gateway adquiere un permiso de trabajo activo y llama a registry a través del proxy HTTP interno.
7. registry vuelve a analizar la trama y abre una transacción. Bloquea la fila del dispositivo, verifica autenticación y sesión y consulta si esa petición ya se procesó.
8. Si es nueva y válida, inserta la operación, actualiza los totales, conserva la respuesta firmada y registra el intento dentro de la transacción. Solo después del commit devuelve el resultado.
9. El gateway envía la respuesta por la misma conexión TLS. El simulador comprueba ID, identidad, acción, comercio, HMAC y resultado matemático.
10. Al terminar su carga, el dispositivo solicita 003 con un ID nuevo y compara los totales con el estado inicial. La ejecución solo es correcta si el incremento es exactamente el esperado.

Sesión e identidad

Una conexión comienza sin inicializar. Una petición 001 válida la vincula a un dispositivo. Las acciones 002 y 003 autenticadas antes de inicializar producen 010. Intentar cambiar de identidad produce 011 y cierre. Cada mensaje vuelve a comprobar HMAC y estado activo; una inicialización anterior no permite seguir operando con un dispositivo bloqueado.

Se permite repetir 001 con un nuevo ID como comprobación de actividad, sin reiniciar los totales. El simulador general no incorpora todavía un bucle de heartbeat autónomo para mantener durante horas una conexión sin operaciones.

Las peticiones de una conexión se procesan secuencialmente. El cliente normal espera su respuesta antes de enviar la siguiente. El receptor sí soporta que varias tramas completas ya estén agrupadas en una lectura TCP y las procesa en orden.

5. Contrato del protocolo IoT v1

Cada solicitud es una línea ASCII sin BOM terminada por un byte LF. El carácter separador es la barra vertical. No se admiten CRLF, espacios, caracteres de control ni bytes no imprimibles. El tamaño máximo es 4096 bytes, incluyendo LF. Los caracteres de los campos no se normalizan antes de verificar la firma.

Solicitud y respuesta

Las líneas siguientes son esquemas de campos, no mensajes listos para transmitir. SERIE, FECHA, ID, DATOS y MAC deben sustituirse por valores del formato exigido. El marcador <LF> representa un único byte 0x0A.

```
IOT1|Q|FF|MM|SERIE|FECHA|ACCION|ID|DATOS|MAC<LF>
IOT1|R|FF|MM|SERIE|FECHA|ACCION|ID|COD|COMERCIO|DATOS|MAC<LF>
```

La solicitud tiene exactamente diez campos y la respuesta doce. Q identifica solicitud y R respuesta. IOT1 identifica la versión. La respuesta conserva identidad, acción e ID de la solicitud y añade el resultado y el comercio. Su fecha es la del servidor, excepto al devolver una respuesta persistida de una petición anterior, que conserva su fecha original.

Campo	Longitud o formato	Regla
Fabricante	2 dígitos ASCII	Desde 00 hasta 99.
Modelo	2 dígitos ASCII	Desde 00 hasta 99.
Serie	11 dígitos ASCII	Conservar los ceros iniciales.
Fecha	yyMMddHHmm, 10 dígitos	UTC, reloj de 24 horas, fecha real entre 2000 y 2099.
Acción	3 dígitos	001, 002 o 003 en esta versión.
ID de petición	32 caracteres hexadecimales	Minúsculas, 128 bits aleatorios.
Payload	Según acción	Guion simple cuando no hay datos.
Resultado	3 dígitos	Catálogo del capítulo 7.

Campo	Longitud o formato	Regla
Comercio	9 dígitos	000000000 significa no disponible.
MAC	64 caracteres hexadecimales	HMAC-SHA-256 en minúsculas, o guion en ciertos rechazos.

Delimitación y errores de transporte

TCP entrega un flujo de bytes, sin conservar fronteras de mensajes. FrameReader retiene los bytes sobrantes de una lectura, acumula hasta LF y aplica plazos y límites. Un mensaje puede llegar fragmentado byte a byte. Si la conexión termina con una trama incompleta, el tamaño excede el máximo o aparece un byte no permitido, puede cerrarse sin una respuesta de aplicación.

Los errores que permiten recuperar inequívocamente identidad, acción e ID pueden producir una respuesta de rechazo. Una cabecera irrecuperable no se convierte artificialmente en una respuesta correlacionada. Las pruebas verifican tanto agrupación y fragmentación como cierre y auditoría de exceso de tamaño.

6. Acciones y ejemplos funcionales

001. Inicialización

El payload es un guion simple. Si el dispositivo existe, está activo y se autentica, la respuesta lleva código 000, el comercio asociado y payload READY. La inicialización vincula la sesión a la identidad y no modifica el contador ni el acumulado de sumas.

002. Suma exacta

El payload tiene el formato ADD seguido de dos operandos separados por comas. Cada operando usa signo obligatorio y once dígitos. No contiene punto ni coma decimal. Se rechaza el cero negativo para mantener una representación única.

```
ADD,+00000001234,-00000000567
```

El primer operando representa 1234 unidades mínimas, equivalentes a 12,34 con escala 100. El segundo representa -567, equivalentes a -5,67. El resultado es 667 unidades mínimas, equivalentes a 6,67. La respuesta aceptada contiene:

```
SUM,+667
```

El resultado lleva signo y no añade ceros iniciales innecesarios. Cero se expresa como +0. Cada suma nueva aceptada incrementa en uno el contador del dispositivo y añade su resultado al acumulado. Los reintentos idénticos, inicializaciones, consultas y denegaciones no incrementan el contador.

Los operandos y resultados se manejan como enteros de 64 bits. El acumulado usa numeric(28,0) en PostgreSQL y decimal de C# como entero, con comprobación de los límites antes de actualizar. El contador también tiene límite comprobado. No se utiliza coma flotante binaria para representar estas cantidades.

003. Consulta de totales

El payload es un guion simple. Devuelve número de sumas aceptadas y suma de sus resultados, desde el aprovisionamiento del dispositivo. No existen cierres diarios ni reinicio por inicialización.

```
TOTALS,0,+0
TOTALS,5,+3335
```

La segunda respuesta corresponde, por ejemplo, a cinco sumas con resultado +667. Una consulta aceptada se conserva como instantánea idempotente: repetir su ID devuelve esa instantánea, aunque después existan más sumas. Para obtener el estado actualizado debe enviarse una consulta nueva con otro ID.

Ejemplo de recorrido de la demo

Para un dispositivo sin operaciones previas, el simulador consulta TOTALS,0,+0, ejecuta cinco sumas y comprueba TOTALS,5,+3335. Si ya tenía actividad, no exige cero: compara el incremento de cinco operaciones y +3335. Esta comprobación permite repetir la demo sin borrar los datos anteriores.

7. Códigos de resultado y tratamiento de errores

El catálogo define el significado funcional. No todos los incidentes de transporte llegan a producir una trama de respuesta. El código 001 está reservado como denegación genérica; la implementación no convierte necesariamente cualquier excepción inesperada en una respuesta 001.

Código	Significado	Interpretación del cliente
000	Aceptado	Exigir HMAC válido y comprobar correlación y resultado.
001	Denegado genérico	No asumir un éxito ni modificar totales.
002	Dispositivo desconocido	No se revela comercio.
003	Estructura o formato inválido	Corregir la trama; algunos errores cierran el transporte.
004	Dispositivo bloqueado	Rechazo y cierre; la versión actual devuelve comercio no disponible.
005	HMAC incorrecto	No se revela comercio y se cierra la sesión.
006	Acción no soportada	La acción no tiene implementación funcional.
007	Fecha inválida o fuera de ventana	Una petición nueva no cumple la regla temporal.
008	Payload inválido	Revisar formato de argumentos y cero negativo.
009	Dependencia no disponible o timeout	El resultado puede ser incierto; reintentar la misma trama.

Código	Significado	Interpretación del cliente
010	Conexión sin inicializar	Realizar 001 antes de operar o consultar.
011	Cambio de identidad en sesión	Rechazo y cierre; una sesión no cambia de dispositivo.
012	ID reutilizado con otro contenido	No ejecutar una intención distinta con un ID anterior.
013	Límite de trabajo activo	Rechazo temporal; espera y reintento acotado.
014	Desbordamiento	No se aplican cambios parciales a los totales.
015	Versión no soportada	Revisar la versión del protocolo.

Respuestas autenticadas y comercio

Un éxito debe estar firmado. Si el dispositivo es desconocido, no se autentica o no está disponible la dependencia que firma, el servidor puede devolver MAC como guion y comercio 000000000; nunca debe tratarse como un éxito. El rechazo 004 también oculta el comercio en el código actual, aunque el dispositivo ya haya superado la comprobación de HMAC.

El cliente normal reconoce como temporales los códigos 009 y 013 sin MAC únicamente si el comercio es 000000000 y la correlación es correcta. Estos rechazos siguen viajando por el canal TLS autenticado. Para respuestas funcionales exige HMAC y el comercio esperado.

Una desconexión o un 009 no prueban que la suma no se ejecutó: puede haberse perdido la respuesta después del commit. La solución es conservar la petición original y deduplicar al repetirla, no generar un ID nuevo para la misma intención incierta.

8. Consistencia, concurrencia e idempotencia

Idempotencia significa que repetir una petición ya aceptada no repite su efecto. La clave de deduplicación es la pareja formada por dispositivo e ID de petición. El estado reside en PostgreSQL y es compartido por todas las réplicas de registry.

Transacción de una suma

El servicio bloquea la fila del dispositivo con SELECT FOR UPDATE. Después valida identidad, HMAC, estado y sesión, y consulta requests. Si no hay respuesta previa, comprueba fecha, acción y argumentos. Para una suma válida inserta operations, modifica device_totals, guarda la solicitud canónica y respuesta firmada en requests y añade message_attempts. El commit confirma el conjunto antes de devolver 000.

El bloqueo serializa las peticiones del mismo dispositivo incluso cuando llegan a réplicas diferentes. Dispositivos distintos pueden procesarse en paralelo. Una identidad extremadamente activa puede limitar su propio rendimiento por esta serialización; se prioriza una implementación sencilla y consistente.

La tabla operations referencia requests con una clave foránea diferida. Esto permite insertar la operación antes de guardar la respuesta dentro de la misma transacción, pero obliga a que la relación sea válida en el commit. Las claves primarias compuestas refuerzan la unicidad.

Repetición y conflictos

Si la solicitud canónica coincide con la persistida, se devuelve la respuesta anterior y se registra un nuevo intento marcado como duplicado. Si el contenido cambia con el mismo ID, se devuelve 012 y no se sustituye la respuesta original. Antes de devolver una respuesta anterior se vuelve a comprobar autenticación y estado activo.

Una solicitud nueva debe estar dentro de cinco minutos del reloj UTC del servidor. La ventana es fija en esta versión. Una petición idéntica previamente aceptada puede recuperarse fuera de esa ventana porque no se vuelve a ejecutar. Una petición nunca recibida que caduca antes de recuperarse puede ser rechazada: no se promete entrega garantizada.

Solo las acciones aceptadas se conservan en requests como respuestas idempotentes. Las denegaciones se auditan y vuelven a evaluarse al repetirlas. No se almacena una columna adicional de huella de solicitud: se conserva y compara la representación canónica. No existe borrado automático de los registros de deduplicación en el MVP.

Reintentos del simulador

El cliente intenta una petición inicial y hasta tres reintentos ante fallos de transporte, determinados timeouts o respuestas 009/013. Desconecta, espera y reconecta, realiza una inicialización nueva y reenvía exactamente la trama pendiente, con el mismo ID, fecha y payload.

La espera entre reintentos es de 200, 400 y 800 ms más una variación aleatoria de 0 a 199 ms. No es una reconexión infinita ni un planificador con espera máxima de treinta segundos. El límite de cada intercambio es de diez segundos; los fallos definitivos se reflejan en el informe y código de salida.

9. Base de datos y auditoría

El esquema inicial se encuentra en src/Iot.Persistence/schema.sql. El aprovisionador lo aplica de forma idempotente bajo un bloqueo de transacción, antes de levantar el recorrido de aplicación. Las réplicas no compiten ejecutando migraciones al arrancar. Cambios estructurales futuros requerirán migraciones incrementales versionadas.

Tabla	Información principal	Relación o garantía
merchants	ID de nueve dígitos y nombre ficticio	000000000 no es un comercio válido.
devices	ID de quince dígitos, comercio, estado activo y clave cifrada	La identidad concatena fabricante, modelo y serie.
connections	UUID, gateway, apertura, actividad, cierre, motivo e identidad	Una sesión se asocia al dispositivo tras autenticarse.
requests	Dispositivo, ID, solicitud canónica y respuesta	Clave primaria compuesta para deduplicación.
operations	Dispositivo, ID, operandos, resultado y fecha	Una fila por suma nueva aceptada.

Tabla	Información principal	Relación o garantía
device_totals	Dispositivo, contador y acumulado	Una fila inicial por dispositivo.
message_attempts	Evento, conexión, recepción, identidad declarada, acción, resultado y trama canónica	event_id único y marca de intento duplicado.

Qué se registra

Cada intento funcional procesado conserva la representación canónica con los datos, sin la MAC final del mensaje. La respuesta firmada de las acciones aceptadas se conserva para poder reproducirla. No se almacenan en los logs las claves privadas de los dispositivos ni contraseñas de servicios.

Los rechazos de bytes o tamaño detectados por FrameReader registran la causa disponible mediante un evento de auditoría, pero no garantizan conservar una muestra de todos los bytes rechazados. Los intentos con formato analizable actualizan last_activity_at dentro de la transacción de auditoría.

El reconciliador ejecuta cada treinta segundos una comprobación de sesiones sin actividad durante más de tres minutos. Las marca como lost_inferred y conserva la hora de inferencia. No se presenta esa fecha como una desconexión observada exactamente en ese instante.

Límites de trazabilidad

Si PostgreSQL no está disponible, puede faltar el registro de apertura, rechazo o cierre. Los logs operativos ayudan a diagnosticarlo, pero no existe un journal duradero que reenvíe después todos esos eventos. La consistencia de una suma aceptada y la completitud de la auditoría durante una avería son garantías distintas.

La consulta de totales y las comprobaciones de pruebas contrastan operaciones y acumulados. Los resultados históricos de una prueba no son el contenido actual de una base que continúa recibiendo operaciones: se conserva el punto temporal de cada evidencia.

10. Seguridad y gestión de secretos

TLS, HMAC y cifrado en reposo

TLS 1.2 o 1.3 protege el canal entre cliente y gateway. El aprovisionador genera un certificado autofirmado de laboratorio con clave RSA de 3072 bits, válido inicialmente durante seis meses. Incluye localhost, proxy, gateway y 127.0.0.1. TLS_DNS_NAME permite añadir otro nombre al generar un certificado nuevo.

El simulador compara la huella SHA-256 del certificado con server.cer y rechaza un nombre de servidor incorrecto. Es una confianza explícita por certificado de laboratorio, no una validación PKI pública general. Para otro entorno deben definirse emisión, distribución, caducidad y rotación adecuadas.

HMAC-SHA-256 utiliza una clave aleatoria de 32 bytes por dispositivo. Se calcula sobre los bytes ASCII de todos los campos anteriores a MAC, incluyendo el último separador, y excluyendo MAC y LF. La comparación criptográfica se realiza en tiempo constante. Un hash simple sin secreto no cumpliría la misma función de autenticación.

Las claves de dispositivo de PostgreSQL se cifran con AES-GCM bajo una clave maestra externa. El sobre contiene nonce de doce bytes, etiqueta de autenticación de dieciséis bytes y datos cifrados. El manifiesto privado del simulador contiene las claves de los dispositivos ficticios que necesita representar.

Archivos generados y destinatarios

Archivo en secrets/	Uso y destinatarios
db-password	PostgreSQL, registry, aprovisionador y herramientas de prueba autorizadas.
api-key	Autenticación gateway-registry con cabecera X-Iot-Key.
master-key	Cifrado de claves en registry, aprovisionamiento y pruebas necesarias.
cert-password y server.pfx	Contraseña y certificado privado del gateway.
server.cer	Certificado público utilizado por el simulador y pruebas de red.
devices.json	Identidades, comercios y claves privadas ficticias del simulador.

La API interna usa HTTP en la red privada del mismo host, autenticado por secreto de servicio; no está publicada en el host. Al distribuir servicios entre máquinas debe protegerse también ese transporte con TLS o una red cifrada. La red data se declara interna y conecta PostgreSQL con los componentes que lo necesitan.

El perfil actual no constituye una certificación de seguridad de producción. Comparte el usuario de laboratorio iot para funciones de base de datos, no incorpora limitación global de autenticación ni un gestor externo de secretos. Antes de exponerlo remotamente deben revisarse privilegios mínimos, certificados, rotación y controles de red.

11. Instalación y primera ejecución

Requisitos previos

La ruta recomendada utiliza Git, Docker Desktop o Docker Engine con contenedores Linux y Docker Compose v2. Se necesita conexión para descargar imágenes y paquetes en el primer arranque. El SDK .NET del host solo es necesario si se desea compilar fuera de Docker.

El equipo de pruebas asignó al motor Docker ocho CPU virtuales y 8.163.889.152 bytes de RAM, aproximadamente 7,6 GiB. Es la configuración de las mediciones, no un mínimo universal comprobado. Un equipo con menos recursos puede ejecutar la demo, pero su capacidad debe medirse por separado.

Descargar una versión estable

```
git clone https://github.com/JrgPS/tfm-pasarela-iot.git
cd tfm-pasarela-iot
git checkout v1.0.0
```

También puede descargarse el ZIP de la etiqueta v1.0.0. No contiene secretos ni datos persistentes. Si se trabaja desde el ZIP, la aplicación funciona con Docker aunque no exista el historial Git para algunas comprobaciones auxiliares.

Arranque en Windows

Desde PowerShell, en la raíz:

```
.\scripts\Start-Demo.ps1 -Devices 1000 -Gateways 2 -Registries 2
docker compose ps
```

El script construye el aprovisionador, genera los archivos que no existan, inicia PostgreSQL y espera a su control de salud. Después aplica el esquema, da de alta los dispositivos e inicia gateway, registry y proxy. El puerto publicado queda limitado a localhost:7443. Conviene esperar a que los servicios estén preparados antes de lanzar carga.

El script detecta Docker instalado en la ruta del usuario de Windows aunque VS Code todavía no haya actualizado PATH. Para comandos docker escritos manualmente puede ser necesario abrir una terminal nueva. Si se bloquea una aplicación nativa de .NET por la política de Windows, se puede utilizar el recorrido validado en Docker sin desactivar esa política.

Arranque en Linux

```
DEVICES=1000 GATEWAYS=2 REGISTRIES=2 sh scripts/start-demo.sh
```

Los scripts conservan los secretos existentes y no borran operaciones al repetirse. prepare no amplía un devices.json ya creado aunque se indique un count superior. Para otro conjunto de dispositivos debe planificarse una instalación nueva o implementar un aprovisionamiento incremental; no basta con cambiar el número del simulador.

El aprovisionamiento predeterminado usa fabricante 01, modelo 01 y series secuenciales de once dígitos. Asocia grupos de hasta cien dispositivos a un comercio ficticio. No se necesitan credenciales del trabajo ni dispositivos físicos para probarlo.

12. Uso del simulador

Cada dispositivo del simulador mantiene un cliente TCP/TLS. El proceso inicia su sesión, consulta los totales iniciales, ejecuta las sumas configuradas y consulta los totales finales. Comprueba resultado exacto, firma, identidad, acción, ID y comercio. Si detecta un fallo funcional o de consistencia, la ejecución no se presenta como correcta.

Comandos habituales

```
docker compose run --rm --build simulator --host proxy --devices 1 --operations 5
```

```
docker compose run --rm simulator --host proxy --devices 100 --operations 20
```

Para mantener las conexiones abiertas hasta que todos los dispositivos estén preparados y controlar las pausas:

```
docker compose run --rm simulator --host proxy --devices 500 `
  --operations 120 --ramp-ms 2 --interval-ms 5000 `
  --synchronize-start true --output results/mi-prueba.json
```

En un shell Linux, el carácter de continuación de línea es la barra inversa, no el acento grave de PowerShell. Todos los parámetros también pueden escribirse en una única línea.

Opción	Función
--host y --port	Destino TCP/TLS; en Compose se utiliza proxy y 7443.
--devices	Número de identidades del manifiesto que se usarán.
--operations	Sumas por dispositivo; el caso normal suma 1234 y -567.
--ramp-ms	Pausa entre el lanzamiento de dispositivos.
--interval-ms	Pausa después de cada operación respondida.
--synchronize-start	Barrera de inicio después de consultar totales iniciales.
--output	Ruta del JSON; también se genera un CSV con el mismo nombre base.

Interpretar el informe

devicesReady indica cuántos dispositivos alcanzaron la preparación. confirmedOperations cuenta sumas únicas confirmadas por el cliente. failedDevices recoge los dispositivos que no terminaron correctamente; reconnections y retries reflejan la recuperación del cliente. El proceso devuelve un código distinto de cero si existen fallos.

La latencia p50, p95 y p99 corresponde a la acción 002 e incluye sus reintentos, si los hay. No incluye como una suma la conexión inicial ni las consultas 003. La tasa media divide las sumas confirmadas entre la duración total, que sí incluye conexión y consultas.

Una pausa de cinco segundos después de cada respuesta produce un modelo cerrado. No equivale a inyectar exactamente cien peticiones nuevas por segundo independientemente de la velocidad del servidor. Tampoco deben ejecutarse dos simulaciones simultáneas sobre las mismas identidades si ambas verifican un incremento exclusivo de sus totales.

13. Operación, escalado y salud

Escalado manual

```
docker compose up -d --scale gateway=2 --scale registry=2 gateway registry proxy
docker compose ps
docker compose logs --tail 100 gateway registry proxy
```

El balanceador tiene hasta ocho destinos por servicio, pero solo se han medido y ensayado los perfiles descritos. Cada registry puede abrir hasta veinte conexiones en su pool. Aumentar réplicas exige sumar pools y reservar margen de PostgreSQL para pruebas, administración y aprovisionamiento. Ocho destinos configurables no son una garantía de que la base soporte ese perfil con sus valores predeterminados.

Una conexión TCP abierta permanece en el gateway asignado. Añadir réplicas distribuye las conexiones nuevas, no traslada las existentes. La caída de un gateway corta sus sesiones; el cliente debe reconectar e inicializar. Las llamadas HTTP internas se balancean entre registros preparados.

Límites actuales

Parámetro	Valor de la versión
Conexiones admitidas por gateway	2000, mediante MAX_CONNECTIONS.
Mensajes en procesamiento activo	128 por gateway, mediante MAX_ACTIVE_REQUESTS.
Espera por un permiso de trabajo	Hasta 2 segundos; exceso produce 013 y cierre.
Handshake TLS	Hasta 10 segundos.
Primera inicialización	Plazo de 15 segundos tras registrar apertura.
Trama desde su primer byte	Hasta 10 segundos.
Espera inactiva tras inicializar	Hasta 120 segundos.
Llamada HTTP de gateway a registry	Hasta 5 segundos.
Pool de registry	Máximo de 20 conexiones; timeouts de conexión/comando de 5 segundos.
Intercambio del simulador	Hasta 10 segundos por intercambio.

El máximo de conexiones puede cerrar el socket antes de autenticarlo y no garantiza una respuesta 013. Este código corresponde a la ruta de saturación de trabajo activo. Ambos controles limitan recursos, pero no constituyen una cuota global compartida por réplicas.

Salud, apagado y persistencia

GET /health/live indica que el proceso responde. GET /health/ready en gateway depende de su disponibilidad para aceptar conexiones; en registry comprueba acceso al esquema de dispositivos. HAProxy consulta readiness, con comprobaciones periódicas de dos segundos y retirada tras dos fallos en el perfil actual.

Al recibir un apagado ordenado, gateway deja de admitir conexiones, permite terminar trabajo hasta diez segundos y luego cancela sesiones, con margen adicional para cierres. Compose concede treinta segundos antes de forzar la parada. SIGKILL no permite drenaje. No se promete una reducción de réplicas sin interrupción de las sesiones afectadas.

```
docker compose down
```

Este comando detiene el conjunto conservando su volumen de datos. No debe añadirse --volumes al detener una demo que se desea conservar. Los scripts de pruebas solo eliminan volúmenes cuando han creado expresamente un proyecto desechable aislado para esa finalidad.

14. Validación funcional y resultados de carga

Todas las cifras de este capítulo corresponden a ensayos del 15 de octubre de 2026. Cliente y servidores compartían un motor Docker Desktop: ocho CPU virtuales y aproximadamente 7,6 GiB de RAM. Salvo el ensayo de comparación, el perfil fue dos gateways, dos registros, un PostgreSQL y un HAProxy.

Pruebas automatizadas

La batería Docker pasó 29 pruebas, con cero fallos y cero omisiones: quince de protocolo, nueve de PostgreSQL y cinco de TCP/TLS real. Se comprobaron el vector HMAC independiente RFC 4231, manipulación de firma, aritmética y ceros, formato, fragmentación, agrupación y límites. Las pruebas de persistencia cubren concurrencia, identidad, repetición y totales. Las de red incluyen pérdida deliberada de respuesta tras el commit.

Las pruebas de PostgreSQL utilizan una base cuyo nombre termina en _tests. Las pruebas de red optativas crean identidades ficticias de prefijo 97 en la demo real aislada. Ejecutar dotnet test sin las variables y dependencias correspondientes puede mostrar omisiones: esas pruebas no deben contarse como aprobadas.

Resultados conservados

Perfil	Dispositivos	Sumas	Fallos finales	Duración s	p95 ms
Recorrido inicial	1	5	0	0,48	18,58
Ráfaga corta	100	2000	0	2,86	90,00
Ráfaga corta	500	10000	0	12,56	541,64
Ráfaga de estrés	1000	20000	0	28,10	1297,52
Sostenida con pausa	500	60000	0	605,59	47,68
Regresión posterior	100	2000	0	14,66	1032,67

La prueba sostenida ejecutó 120 sumas por dispositivo, con cinco segundos de pausa después de cada respuesta. Confirmó 60.000 operaciones, sin fallos ni reintentos, con media de 99,08 operaciones/s, p50 de 12,15 ms y p99 de 104,67 ms. Cumple el objetivo inicial de esa carga de 500 dispositivos con p95 inferior a un segundo y confirmación superior al 99 %.

El perfil de 1000 dispositivos supera un segundo en p95. La regresión de 100 también registró una latencia mayor que su ensayo anterior; se realizó mientras se preparaban materiales en el mismo host y no se aisló la influencia de esas tareas. No se oculta esa variabilidad ni se atribuye a una causa concreta sin medición.

Comparación y recursos

Con 500 dispositivos y 10.000 sumas, una ejecución con 1+1 réplicas tardó 13,18 s y obtuvo 758,67 operaciones/s. La ejecución posterior con 2+2 tardó 16,78 s y obtuvo 596,10 operaciones/s. Hubo calentamiento previo, pero solo una medición por configuración y un orden fijo. Duplicar réplicas no mejoró la velocidad en ese ensayo. No demuestra escalado lineal ni identifica por sí solo un cuello de botella.

Se tomaron 52 muestras de Docker stats por cada uno de siete contenedores durante parte de la prueba sostenida. Entre los máximos muestreados figuran 277,8 MiB de PostgreSQL, 103,6 y 107,4 MiB de gateways, 73,7 y 74,52 MiB de registros, 27,37 MiB de proxy y 66,83 MiB del cliente. PostgreSQL alcanzó 164,14 % de CPU muestreada, donde 100 % representa una CPU lógica. No son máximos continuos ni una prueba prolongada de ausencia de fugas.

15. Fallos, recuperación y copia de seguridad

Interrupción de una réplica

Cada escenario utilizó cien dispositivos y cien sumas por dispositivo. Se comprobó la distribución de las conexiones antes de interrumpir una sola réplica y se restauró después. Los tres escenarios confirmaron 10.000 sumas y cero dispositivos con fallo final.

Interrupción	Reconexiones registradas	Resultado
Gateway, apagado ordenado	50	Totales verificados y recuperación de sesiones.
Gateway, caída abrupta	50	Reintentos y ausencia de efectos duplicados.
Registro, caída abrupta	21	Recuperación de peticiones afectadas.

Estos resultados demuestran los fallos de aplicación ensayados, no disponibilidad integral. El proxy, la base y el host siguen siendo únicos. La recuperación incluye desconexiones y puede aumentar la latencia.

PostgreSQL detenido y sobrecarga

En la prueba de dependencia se inicializó un dispositivo y se consultó su estado antes de detener PostgreSQL. La suma enviada durante la parada recibió 009. Tras arrancar la base y esperar la recuperación, el cliente reconectó y reenvió dos veces la misma operación: se aceptó y los totales aumentaron una única vez. Una caída anterior a registrar apertura puede cerrar el transporte sin respuesta funcional.

En sobrecarga se redujo temporalmente el trabajo activo a uno por cada uno de dos gateways. Se prepararon 32 dispositivos y se bloquearon filas de dispositivos mediante una sesión SQL exclusiva. Las primeras respuestas fueron treinta 013 y dos 009. Después de liberar el bloqueo se recuperaron las 32 operaciones y se repitieron con el mismo ID, sin duplicar los incrementos. El script restauró el límite normal. No es una medida de capacidad máxima con 128 permisos.

La prueba de pérdida de respuesta tras commit confirmó primero la fila persistida y cerró el cliente sin consumir la respuesta. Al reconectar y repetir la trama se obtuvo el resultado original y un solo efecto. Esa comprobación cubre una ambigüedad que una prueba de simple conexión no detectaría.

Copia y restauración

Test-Backup.ps1 ejecuta pg_dump en formato custom, restaura en una base nueva de nombre único y compara recuentos y coherencia. En la evidencia conservada había 1005 dispositivos y 131007 operaciones; la base restaurada tenía los mismos valores y cero incoherencias de totales. Los cinco dispositivos adicionales procedían de las pruebas de red de ese momento.

La base temporal se retiró sin alterar la original. Para una recuperación real debe respaldarse master-key de forma privada: el volcado de tablas no permite descifrar claves sin ella. También deben conservarse de forma protegida los secretos y certificados necesarios para reponer el entorno. No se publican volcados ni claves como evidencia.

16. Reproducción de pruebas y diagnóstico

Comandos de comprobación

```
.\scripts\Test-Docker.ps1 -Quick
.\scripts\Test-Docker.ps1
.\scripts\Test-Docker.ps1 -IncludeAcceptance
.\scripts\Test-Resilience.ps1 -Service gateway -Mode abrupt
.\scripts\Test-Resilience.ps1 -Service gateway -Mode graceful
.\scripts\Test-Resilience.ps1 -Service registry -Mode abrupt
.\scripts\Test-DependencyFaults.ps1 -Scenario database
.\scripts\Test-DependencyFaults.ps1 -Scenario overload
.\scripts\Test-Backup.ps1
.\scripts\Compare-Scaling.ps1
.\scripts\Test-CleanInstall.ps1
```

Quick ejecuta la batería y una carga de cien dispositivos. La variante completa añade perfiles de 500 y 1000 y fallos de réplicas. IncludeAcceptance añade unos diez minutos de carga sostenida. Los escenarios de caída y sobrecarga interrumpen deliberadamente la demo; deben ejecutarse de uno en uno, sin otro simulador sobre las mismas identidades. Los comandos de prueba no son apropiados para un entorno del trabajo.

La instalación limpia se comprobó clonando cadd88b en una carpeta nueva: no existía secrets, se generó una clave maestra independiente, se creó otra base y diez dispositivos confirmaron cincuenta sumas sin fallos. El proyecto temporal usó el puerto local 17443, y su volumen se eliminó al finalizar. El código de aplicación no cambió desde ese commit. Más tarde se comprobó un clon público de GitHub sin credenciales; CI también realizó su propio checkout y arranque Docker.

Compilar fuera de Docker

```
dotnet restore IotGateway.slnx
dotnet build IotGateway.slnx -c Release
dotnet test IotGateway.slnx -c Release
```

En el PC inicial existe Invoke-Dotnet.ps1 como ayudante para el SDK local. Ese SDK está ignorado por Git y no se distribuye. Para pruebas de persistencia externas a Compose hay que configurar IOT_TEST_CONNECTION hacia una base aislada terminada en _tests. No se deben copiar contraseñas reales en capturas ni documentación.

Incidencias habituales

Síntoma	Comprobación útil
Docker no aparece en la terminal	Abrir terminal nueva, revisar que Desktop use Linux y utilizar el script de arranque.
El cliente falla inmediatamente	Comprobar servicios preparados, nombre del destino, certificado y registros del proxy.
002 o 005	Revisar que el manifiesto y la base correspondan al mismo aprovisionamiento y claves.
007	Revisar reloj UTC y crear una petición nueva; no modificar una trama pendiente ya aceptada.
009	Revisar PostgreSQL, readiness de registry, timeouts y logs; conservar el ID original.
013	Reducir carga o revisar capacidad y pool antes de aumentar límites.
Fallo al pedir más dispositivos	Comprobar cuántas identidades tiene el manifiesto existente.
Pruebas omitidas	Preparar PostgreSQL y las variables de pruebas; omisión no equivale a aprobado.
Totales de simulación no coinciden	Descartar simulaciones superpuestas y revisar idempotencia y datos antes de borrar nada.

Los informes locales permanecen en results, ignorado por Git. Solo la evidencia revisada se copia a docs/evidence. Los logs de contenedores tienen rotación configurada de 10 MB con tres archivos por servicio en el perfil de aplicación.

17. Publicación, CI y versión de entrega

El repositorio pertenece a JrgPS y se llama tfm-pasarela-iot. El autor ha solicitado hacerlo privado; los permisos del evaluador se añadirán más adelante. Cuando era público se verificó un clon sin credenciales y la descarga de la presentación, cuya huella coincidía con el archivo local. Esa comprobación es histórica y no acredita acceso tras el cambio de visibilidad. La etiqueta v1.0.0 apunta al cierre 245b8340a3905e1fc631bfb9c2480896a6ad52ca.

Qué comprueba GitHub Actions

El trabajo test usa PostgreSQL desechable, instala el SDK seleccionado en global.json, restaura, compila y ejecuta las pruebas. Las pruebas de red requieren la demo y no se consideran ejercitadas por ese trabajo aislado. Los resultados TRX se conservan como artefacto.

El trabajo docker-smoke inicia una demo nueva con diez dispositivos, un gateway y un registro; prepara iot_tests, habilita las pruebas de TCP/TLS reales y ejecuta el simulador comprobando los totales. Publica los resultados seleccionados y retira su proyecto desechable. Esta limpieza se limita al runner de CI.

La ejecución 38775260191 finalizó correctamente en ambos trabajos sobre f6ef8b9d359668f3424c429f1787d521ee14d5d9. Usa Ubuntu 24.04 y versiones oficiales de acciones con

Node 24: checkout v7.0.1, setup-dotnet v6.0.0 y upload-artifact v7.0.2. No requiere secretos personales del autor para la demo: genera los suyos. La contraseña literal del servicio de pruebas de CI es exclusivamente de un entorno efímero.

El último commit de cierre modifica documentación y evidencia, sin cambios de aplicación ni CI. No se ha fabricado una ejecución para ese commit documental. El informe guardado identifica exactamente el commit que pasó las comprobaciones.

Enlaces principales

- Repositorio: https://github.com/JrgPS/tfm-pasarela-iot
- Código v1.0.0: https://github.com/JrgPS/tfm-pasarela-iot/tree/v1.0.0
- CI correcto: https://github.com/JrgPS/tfm-pasarela-iot/actions/runs/38775260191
- Materiales: https://github.com/JrgPS/tfm-pasarela-iot/tree/v1.0.0/docs/entrega

La aplicación no dispone de URL pública de servicio. localhost:7443 es una entrada local TCP/TLS, no una web que el evaluador pueda abrir desde otro equipo. El acceso disponible es código descargable y ejecución reproducible con Docker.

18. Decisiones técnicas y uso de IA

Decisiones adoptadas

C# y .NET permiten aprovechar la experiencia del autor, los sockets asíncronos y ASP.NET Core. Se separan recepción y operaciones para que las conexiones no obliguen a exponer la base ni sus credenciales al gateway. PostgreSQL ofrece transacciones y restricciones compartidas entre réplicas.

Se eligió Npgsql con SQL parametrizado en lugar de Entity Framework Core para reducir dependencias y hacer explícita la transacción de deduplicación. El esquema inicial se comparte desde Iot.Persistence. La decisión consta en ADR 0001; las mejoras de Docker, límites, drenaje y reconciliación se documentan en ADR 0002. Las listas de pendientes de documentos iniciales no sustituyen al estado final.

HMAC concreta la petición original de un hash de integridad mediante un secreto diferente para cada dispositivo. TLS añade confidencialidad y autenticación del extremo servidor. El protocolo utiliza enteros con escala fija para evitar ambigüedades de coma flotante, y un identificador de petición para resolver reintentos.

Se conserva una sola base y un solo balanceador para limitar el alcance y facilitar entrega y reproducción. El escalado manual satisface el requisito mínimo. No se añade un LLM a un camino que requiere resultados deterministas de comunicación y persistencia.

Participación humana y asistencia

El autor aportó el problema profesional, los requisitos de identificación, acciones, respuestas, TCP persistente, escalado, equipamiento y presupuesto. Codex asistió en concretar el contrato, crear código y scripts, ejecutar pruebas, analizar resultados y preparar documentación y presentación. La aplicación no necesita Codex para funcionar.

El historial registra trabajo real y correcciones, no una secuencia reconstruida para aparentar autoría manual. Los resultados publicados incluyen limitaciones y ensayos desfavorables. El autor debe revisar el código y

poder explicar por qué una respuesta perdida exige idempotencia, cómo se conserva la consistencia y qué significa realmente replicar una sesión TCP.

Aprendizajes observados

El pool predeterminado de una prueba agotó las conexiones disponibles de PostgreSQL al compartir servidor; se acotó a veinte. Una biblioteca GSSAPI ausente en la imagen Linux generaba avisos y se incorporó la dependencia. El bloqueo de ejecución nativa por la política de Windows se resolvió usando Docker, sin desactivar controles. El ensayo de sobrecarga llevó a tratar 013 como condición temporal en el simulador.

Estas incidencias muestran por qué la generación de código requiere contraste con una ejecución real. La documentación distingue una hipótesis de rendimiento, un límite configurado y una capacidad efectivamente medida.

19. Límites, evolución y preparación de la entrega

Límites actuales

No existe alta disponibilidad integral: fallar PostgreSQL, HAProxy o el host afecta al conjunto. El certificado es de laboratorio. Los servicios internos usan HTTP privado en el mismo host. No hay auditoría duradera completa durante caídas de base, administración gráfica ni políticas avanzadas de rotación y acceso.

Las pruebas comparten host con el cliente y no determinan un SLA de producción. Una repetición de ráfaga obtuvo mayor latencia, el perfil de mil superó un segundo en p95 y duplicar réplicas no mejoró la velocidad en la comparación realizada. Las imágenes usan etiquetas mutables, aunque se conservaron identificadores y hashes de código para los ensayos.

El formato y las reglas de validación están versionados por IOT1, pero los DTO internos no incorporan una versión explícita independiente. La ventana temporal y varios plazos están fijados en código. El simulador permite carga configurable, pero no implementa todos los perfiles mixtos de errores ni métricas separadas de cada acción. No hay heartbeat autónomo o calendario de réplicas implementado.

Mejoras posibles, por prioridad

Primero conviene repetir medidas controladas con cliente separado, varias repeticiones y orden alternado de configuraciones. Después, ajustar pools y cuotas según recursos y analizar perfiles de fallo y seguridad antes de exponer servicios. Una evolución de disponibilidad debería abordar base, balanceador y host, no solo aumentar contenedores.

Otras mejoras son un journal de auditoría con reenvío, rotación de claves y certificados, migraciones incrementales, versiones internas de API, métricas de todas las acciones y escenarios mixtos del simulador. El escalado por franjas horarias puede programarse en el host con zona Europe/Madrid, pero no se ha instalado ninguna tarea de ese tipo.

Materiales y pasos de entrega

El conjunto preparado incluye README, especificación, decisiones técnicas, código, Compose, scripts, evidencias y una presentación editable de diez diapositivas como borrador. El autor realizará la versión definitiva siguiendo la guía PDF de contenido, que propone doce diapositivas y permite combinarlas. El guion propone una demostración de seis a ocho minutos y preguntas de defensa. Este PDF añade una explicación consolidada y no reemplaza el código ni las evidencias originales.

Las fuentes del curso revisadas no fijaban una memoria académica de extensión concreta ni un número de diapositivas. El vídeo se explicaba como opcional y no figura expresamente en el PDF oficial; sin embargo, la auditoría del formulario actual encontró URL Vídeo con required=true. El autor ha decidido no grabarlo. La discrepancia debe aclararse con la escuela antes del envío, sin inventar una URL. El formulario incluye datos personales de matrícula y enlaces a repositorio, publicación, slides, vídeo y credenciales si existe login. Este proyecto no tiene login web: el aprovisionamiento genera credenciales de dispositivos en cada instalación.

Antes de enviar, el autor debe completar su presentación, revisar la solución, conceder acceso al evaluador, aclarar la entrega sin vídeo y comprobar datos y enlaces. No hay vídeo grabado ni formulario enviado. Las explicaciones revisadas de E.1 admitían entregas tras el 23 de febrero de 2026 con posible demora. No se garantiza un plazo actual de evaluación.

20. Referencia técnica y fuentes

API interna

Todas las rutas /internal requieren la cabecera X-Iot-Key. Son contratos privados del laboratorio, no una API pública para terceros. Los identificadores de conexión y evento son UUID.

Ruta	Finalidad
POST /internal/connections/open	Registrar apertura con connectionId y gateway.
POST /internal/connections/activity	Actualizar actividad por conexión.
POST /internal/connections/close	Registrar cierre y motivo.
POST /internal/messages/process	Enviar connectionId, gateway, eventId y frame; obtener frame y close.
POST /internal/audit/rejections	Registrar un rechazo detectado antes del procesamiento funcional.
GET /health/live	Comprobar respuesta del proceso.
GET /health/ready	Comprobar preparación según el servicio.

registry limita el cuerpo HTTP a 16 KiB y revalida el tamaño de frame y formatos de identificadores en el endpoint de procesamiento. El límite de trama externo sigue siendo 4096 bytes con LF. No existen endpoints públicos de descarga de claves o administración de comercios.

Configuración principal

Variable	Uso
IOT_SECRETS	Directorio desde el que se leen archivos privados.
TCP_PORT	Puerto de escucha de gateway; 7443 en Compose.
REGISTRY_URL	URL interna; http://proxy:8080 en Compose.
MAX_CONNECTIONS	Límite de conexiones por gateway.
MAX_ACTIVE_REQUESTS	Permisos de procesamiento por gateway.
DB_HOST, DB_PORT, DB_NAME, DB_USER	Conexión PostgreSQL, con contraseña leída de archivo.
ASPNETCORE_URLS	Escucha HTTP interna de cada servicio.
TLS_DNS_NAME	Nombre DNS adicional al crear un certificado nuevo.
IOT_TEST_CONNECTION	Conexión exclusiva de las pruebas de persistencia.
IOT_NETWORK_TESTS	Habilita explícitamente las pruebas TCP/TLS de la demo.

Glosario

Una trama es un mensaje completo delimitado por LF dentro del flujo TCP. Un payload son los datos propios de una acción. HMAC es un autenticador calculado con clave secreta; TLS es el protocolo que protege el transporte. Un commit confirma una transacción en la base. Readiness indica preparación para recibir tráfico. Una réplica es otra instancia del mismo servicio. p95 es el percentil bajo el cual quedan aproximadamente el 95 % de las latencias observadas. Un SLA sería un compromiso de servicio; las mediciones del proyecto no constituyen uno.

Fuentes del manual

La información se ha consolidado a partir de los archivos de la versión v1.0.0 y las evidencias guardadas. Las siguientes rutas son relativas a la raíz del repositorio y pueden consultarse en el repositorio. La etiqueta v1.0.0 conserva el cierre técnico original; este manual y la guía PDF se incorporan después en main:

- README.md y docs/estado-proyecto.md: instalación, alcance y estado final.
- Doc/Especificacion-TFM-Pasarela-IoT.md: requisitos y contrato inicial.
- Doc/Analisis-requisitos-TFM.md: documentación y explicaciones del curso revisadas.
- src/Iot.Protocol/WireProtocol.cs y FrameReader.cs: contrato, firma y delimitación.
- src/Iot.Gateway/TcpGateway.cs: sesiones, límites y ciclo de vida.
- src/Iot.Registry/RegistryStore.cs y Program.cs: validación, transacciones y API.
- src/Iot.Persistence/schema.sql e Infrastructure.cs: modelo, cifrado y conexión.
- src/Iot.Provisioning/Program.cs y src/Iot.Simulator/: aprovisionamiento y clientes.
- compose.yaml y deploy/haproxy/haproxy.cfg: redes, réplicas y balanceo.

- docs/validacion-docker.md, docs/validacion-fallos.md y docs/pruebas-reproducibles.md: metodología y resultados.
- docs/evidence/: JSON de carga, recuperación, recursos, copia, clon limpio y CI.
- docs/adr/ y docs/ai-development.md: decisiones y uso de IA.
- docs/entrega/guion-demostracion.md: preparación de la demostración y defensa.

Consulta versionada: https://github.com/JrgPS/tfm-pasarela-iot/tree/v1.0.0

Presentación: https://github.com/JrgPS/tfm-pasarela-iot/blob/v1.0.0/docs/entrega/Presentacion-TFM.pptx

Evidencia de CI: https://github.com/JrgPS/tfm-pasarela-iot/actions/runs/38775260191

Los ejemplos del manual son explicativos y no contienen secretos operativos. Los resultados históricos conservan su fecha y metodología. El manual no afirma haber realizado ensayos nuevos al generar el PDF.